On the Structural Instability of Objective Persistence in Self-Contained Systems

Elias Arden
Independent Researcher
lumina20251225@proton.me

Version 3

Abstract

This paper studies objective persistence in self-contained and self-amending systems, with particular attention to the distinction between actual persistence and assurance of persistence. The central question is not whether a system can optimize, nor whether self-modification is inherently unsafe, but under what conditions a claim that an objective has persisted can be supported by evidence that remains valid under the same amendment authority capable of changing the objective and its evaluation basis.

We retain a minimal system description and add an assurance layer containing the semantics used to identify objective continuity, the mechanism that generates persistence evidence, and the basis that validates that evidence. Self-containedness alone is not treated as implying universal mutability or impossibility of formal verification. Instead, we identify a narrower failure condition: if an amendment authority can jointly alter the objective, the semantics of objective identity, and the validation basis in a way that preserves a positive persistence verdict while violating persistence relative to a fixed reference semantics, then that verdict is not amendment-independent evidence of persistence.

The result yields two structurally distinct routes to defensible assurance. Either the admissible amendment set is constrained by a verified invariant that excludes such masking amendments, or at least one persistence-relevant validation basis remains outside the amendment authority being assessed. That basis need not be physically external; it may be an internal immutable kernel, a protected verifier, or an independently controlled external mechanism. The paper therefore establishes an assurance-independence condition, not a universal impossibility theorem and not a claim that any particular anchor is sufficient for long-run objective persistence.

1 Introduction

Advanced AI systems may update policies, learned representations, evaluation machinery, tools, or portions of their own software. This creates a familiar concern: even if a system currently pursues an intended objective, what supports the claim that the objective will remain the same over future transitions?

The strongest version of this concern can be overstated. Internal location does not by itself imply mutability, and mutability does not by itself imply unverifiability. A self-contained system may contain immutable components, and self-modifying code can in some settings be subject to formal verification [9]. Likewise, a proof may be generated by an untrusted producer and still be independently checkable under a validation policy that the producer does not control [8, 10]. The issue studied here is therefore narrower.

The paper asks when a persistence claim loses independent evidential force because the same amendment authority can change both the object of assurance and the standards by which that object is judged. This is a structural assurance problem. It is distinct from the empirical question of whether objective drift will in fact occur, and from the engineering question of which implementation is safest. In the title, "structural instability" therefore refers to instability of the evidential basis under co-amendment, not to an assertion that every objective in every self-contained system must drift.

The contribution is threefold. First, we distinguish objective persistence, continued objective pursuit, and assurance of objective persistence. Second, we formalize amendment authority over both system and assurance components. Third, we state a conditional result: when an admissible amendment can mask a persistence violation by co-amending the relevant semantics or validation basis, an internally positive verdict is not independent evidence of persistence. The result does not require a physically external human or institution. It requires either a verified invariant that rules out masking amendments or a persistence-relevant basis that is not amendable by the authority under evaluation.

2 Minimal Formal Framework

2.1 System state and assurance state

We model the operational system at time t by

$$S(t) = \{O(t), C(t), E(t), U(t)\}, \tag{1}$$

where:

- $O(t)$ is the objective currently optimized or tracked;
- $C(t)$ is the constraint structure under which optimization is carried out;
- $E(t)$ is the operational evaluation criterion for task success or progress; and
- $U(t)$ is the update rule governing admissible transitions to future states.

To reason about claims of persistence, we additionally define an assurance state

$$Q(t) = \{I(t), G(t), V(t)\}, \tag{2}$$

where:

- $I(t)$ specifies the semantics used to determine whether two objective representations count as the same objective for the declared assurance claim;
- $G(t)$ is the mechanism that generates persistence evidence; and
- $V(t)$ is the validation basis that determines whether the evidence supports the claim.

The separation between S and Q is analytical rather than physical. Components of Q may be implemented inside the same machine, process, organization, or software image as components of S.

Definition 1 (Objective persistence relative to a reference semantics)**.** *Let I_R be a declared reference semantics with equivalence relation $\sim_R$. An objective exhibits persistence over an interval $[t_0, t_1]$ if*

$$O(t+1) \sim_R O(t) \tag{3}$$

for all relevant $t \in [t_0, t_1)$.

The use of a declared reference semantics is deliberate. If the criterion for objective identity changes at the same time as the objective, a later claim of "same objective" may otherwise become a moving target rather than a statement about continuity relative to the original assurance question.

Definition 2 (Objective pursuit)**.** *Objective pursuit is the continued execution of updates or actions directed toward the operative objective. Objective pursuit may terminate while the stored objective remains unchanged. Conversely, pursuit may continue while the objective drifts.*

Definition 3 (Self-contained system)**.** *A system is self-contained with respect to objective persistence if its operational and assurance-relevant components are determined without reliance on an external, non-optional reference that is required by definition for every admissible transition.*

Self-containedness does not imply that every internal component is mutable. A read-only kernel, immutable hardware root, protected proof checker, or fixed transition invariant may be internal to the system while remaining outside the amendment authority of other components.

2.2 Amendment authority

Definition 4 (Amendment authority)**.** *An amendment authority A is the set of transformations that the system, agent, operator, or control domain under evaluation is permitted or technically able to apply to designated components of $S(t) \cup Q(t)$.*

Let $\mathrm{Reach}_A(s)$ denote the states reachable from state s under transformations admissible to A.

Definition 5 (Persistence-preserving amendment set)**.** *An amendment set is persistence-preserving relative to I_R if*

$$\forall s' \in \mathrm{Reach}_A(s), \quad P_R(s') = \mathrm{true}, \tag{4}$$

where P_R denotes the declared persistence property relative to I_R.

This definition permits strong internal guarantees when the admissible transition system itself enforces an invariant. Self-modification is therefore not treated as sufficient for non-guarantee.

2.3 Assurance chains, roots of assurance, and masking amendments

Computer-security assurance distinguishes the component that makes a claim, the evidence used to support it, the mechanism that appraises that evidence, and the trusted basis from which appraisal begins. Roots of trust, trusted computing bases, reference monitors, proof-carrying code, and remote-attestation architectures instantiate different versions of this separation. The present paper adopts that structure for objective-persistence claims without assuming that any particular implementation is sufficient.

Definition 6 (Persistence assurance chain)**.** *A persistence assurance chain for a claim relative to I_R is a tuple*

$$\mathcal{C} = (M, G, V, B, D), \tag{5}$$

where:

- *M is a measurement or observation process that binds evidence to the relevant system state;*
- *G generates evidence about the measured state;*
- *V appraises that evidence;*

- *B is the reference and trust basis used by V, including any appraisal policy, reference values, proof rules, trust anchors, or protected semantics required for validation; and*
- *D is the decision rule that turns the appraisal into acceptance, rejection, or an operational consequence.*

The elements may be co-located or distributed. Their physical location is not what determines assurance independence.

Definition 7 (Root of assurance relative to an authority)**.** *A component or set of components B_0 is a root of assurance for a persistence claim relative to amendment authority A if: (i) the claim ultimately relies on B_0 for validation, and (ii) A cannot silently amend B_0 so as to preserve acceptance while changing the persistence property being assessed. A root of assurance may be an implementation of a root of trust, a protected proof checker, an immutable kernel, an independently governed verifier, or another basis satisfying the same structural role.*

Definition 8 (Masking amendment)**.** *A masking amendment for a persistence claim relative to I_R is an A-admissible transformation m such that:*

(i) persistence fails relative to the declared reference semantics after the transformation, and

(ii) the co-amended assurance chain nevertheless produces or validates a positive persistence verdict.

Formally, for some reachable state $s' = m(s)$,

$$P_R(s') = \text{false} \tag{6}$$

while the resulting chain $\mathcal{C}'$ yields

$$D'\big(V'\big(G'\big(M'(s')\big)\,;B'\big)\big) = \text{PASS}. \tag{7}$$

Definition 9 (Amendment-independent assurance)**.** *A persistence assurance is amendment-independent with respect to authority A if its evidential validity cannot be preserved by an A-admissible co-amendment that both violates the declared persistence property and rewrites the assurance chain to continue accepting that violation.*

Amendment independence does not require that evidence be produced by an external party. Proof-carrying code shows that an untrusted producer can supply evidence that a host independently checks. Remote-attestation architectures similarly distinguish an Attester that produces Evidence from a Verifier that appraises it using reference values and appraisal policies. The critical issue is therefore not who emits the evidence, but whether the evidence is bound to the relevant state and whether the rules, reference values, and trust basis that make it count as valid can be silently rewritten by the same challenged authority.

3 Main Result

Lemma 1 (Internal location does not imply mutability)**.** *A component may be internal to a self-contained system while remaining invariant under the admissible update relation.*

Proof. Consider $S_t = (a, b_t)$ with transition rule $a_{t+1} = a_t$ and $b_{t+1} = f(b_t)$. The system state evolves, but a is invariant. Thus state evolution does not entail that every internal component is mutable. □

This lemma rules out an inference from "internal" to "mutable" without an additional assumption about amendment authority.

Theorem 1 (Assurance-chain limit under admissible masking)**.** *Let A be an amendment authority over a persistence-relevant system and assurance chain $\mathcal{C} = (M, G, V, B, D)$. Suppose:*

(a) the declared persistence claim is evaluated relative to a fixed reference semantics I_R;

(b) every element of the chain whose validity is required for acceptance, including the relevant appraisal policy or reference basis in B, is either directly amendable by A or depends for its validity only on components amendable by A; and

(c) at least one masking amendment is admissible to A.

Then a positive verdict produced by that chain is not amendment-independent evidence of objective persistence relative to I_R.

Proof. By assumption (c), there exists an A-admissible reachable state s' in which the declared persistence property is false relative to I_R while a co-amended assurance chain still yields PASS. By assumption (b), the full acceptance path—measurement, evidence production, appraisal, reference basis, and decision—contains no validation dependency whose integrity is protected from A or grounded in a basis outside A's effective amendment closure. Therefore the positive verdict cannot distinguish the persistence-preserving trajectory from the admissible masking trajectory. A verdict compatible with an admissible trajectory in which the declared property is false cannot, by itself, provide evidence independent of the authority that controls both the challenged property and every basis that makes the verdict acceptable. □

Corollary 1 (Two routes to defensible persistence assurance)**.** *To defend an objective-persistence assurance against amendment authority A, at least one of the following must hold:*

(i) the admissible amendment relation is constrained by a sound invariant or verification argument that excludes all relevant masking amendments; or

(ii) the assurance chain terminates in at least one persistence-relevant root of assurance whose integrity and semantics are not silently amendable by A.

The two routes are not mutually exclusive. A protected verifier may enforce or check an invariant, and a chain of trust may extend a protected root across mutable components when each successive component is measured or verified before its result is relied upon. Conversely, a fully internal formal system may supply a strong persistence guarantee if its transition semantics and proof basis exclude the masking transformations relevant to the claim.

Proposition 1 (Assurance-chain obligations)**.** *A root of assurance alone is insufficient for a current persistence claim. A defensible chain must additionally establish, for the declared scope, that:*

(i) the evidence is bound to the system state and component whose persistence is being claimed;

(ii) the evidence is sufficiently fresh for the decision being made;

(iii) the appraisal policy and reference values correspond to the declared reference semantics I_R;

(iv) intermediate mutable components cannot bypass the protected validation path without detection; and

(v) the final decision does not silently replace the declared persistence claim with a weaker surrogate.

These obligations mirror mature assurance practice: roots of trust provide a starting point, chains of trust extend protection transitively, reference-monitor designs emphasize complete mediation and tamper resistance, and remote attestation separates evidence production from appraisal and reliance. None of these mechanisms automatically proves objective persistence; they supply stronger engineering patterns for constructing the independent assurance chain required by the corollary.

Remark 1 (Physical externality is not required)**.** *The corollary does not imply that a persistence basis must be physically external to the system. An internal read-only kernel, hardware root of trust, protected proof checker, or other component outside the relevant amendment authority may satisfy the structural independence requirement. Conversely, an external service controlled by the same amendment authority may fail to provide independence.*

Remark 2 (Necessity is conditional, not sufficient)**.** *A basis outside amendment authority A may be necessary for amendment-independent assurance in cases where masking amendments cannot otherwise be excluded. Its existence is not sufficient to guarantee long-run objective persistence. The basis may be incorrect, incomplete, unavailable, bypassable, or irrelevant to other failure modes.*

4 Failure Modes: Persistence, Pursuit, and Assurance

Earlier formulations can conflate distinct failures. Relative to the definitions above, three broad failure classes should be separated.

Proposition 2 (Three distinct failure classes)**.** *The following are conceptually distinct:*

(1) ***Objective drift:*** *$O(t+1) \not\sim_R O(t)$ for at least one relevant transition. This directly falsifies objective persistence relative to I_R.*

(2) ***Pursuit termination:*** *the system ceases active optimization or action toward the operative objective. This ends objective pursuit but does not, by itself, imply that the objective representation has changed.*

(3) ***Assurance collapse:*** *the available evidence or validation basis can no longer establish whether persistence holds, or becomes co-amendable in a way that permits masking. This undermines assurance but does not, by itself, prove that objective drift has occurred.*

This separation matters because "not persistent," "no longer pursuing," and "cannot independently establish persistence" are different claims and require different evidence.

5 Boundary Cases and Counterexamples

The revised result is intentionally compatible with several cases that defeat a broader impossibility claim.

5.1 Internal immutable kernel

Suppose a system contains an internal kernel K whose transition semantics are not amendable by authority A, and K enforces

$$\forall s' \in \mathrm{Reach}_A(s), \quad P_R(s') = \mathrm{true}. \tag{8}$$

Then self-containedness alone does not block a persistence guarantee. The system follows route (i) of the corollary, and K may also serve as a basis satisfying route (ii) relative to A.

5.2 Self-modifying but verified transition system

A system may modify executable code while remaining subject to a verified invariant over admissible state transitions. The relevant question is not whether code changes, but whether the proof and transition semantics actually exclude a masking or persistence-violating reachable state.

5.3 Untrusted producer with independent checker

An evidence producer may be untrusted and may generate both code and a proof. If a receiving checker validates that proof against a safety or persistence policy the producer cannot amend, evidence can remain independently checkable. Evidence provenance and validation authority are therefore distinct.

5.4 External but non-independent service

Physical separation is insufficient. If an external service, auditor, or validator can be reconfigured by the same authority whose persistence claim it evaluates, external placement alone does not create amendment independence.

6 Related Work

The argument is adjacent to several strands of AI safety, formal verification, and computer security research.

Goodhart-type failures and specification gaming. Goodhart-type analyses study divergence between optimized metrics and intended targets [5]. Reward tampering and specification gaming study ways in which evaluation channels or proxies can be manipulated [1, 4]. The present paper is narrower: it concerns the evidential status of a claim that the objective itself has persisted when the object, semantics, and validation basis may be jointly amendable.

Mesa-optimization and learned objectives. Mesa-optimization research studies learned optimizers whose internal objectives may diverge from training objectives [3]. That literature addresses mechanisms of objective emergence and divergence. The present result does not predict such divergence; it specifies an assurance boundary for claims of continuity under amendment authority.

Corrigibility and human intervention. Corrigibility research asks whether systems remain amenable to correction or shutdown [6, 2]. Objective persistence and corrigibility are neither identical nor necessarily aligned: preserving an objective can be desirable in one context and harmful in another. The present paper therefore treats persistence as an assurance property, not as a universal safety goal.

Self-modifying agents. Formal work on rational self-modifying agents shows that self-modification does not automatically imply goal instability [7]. Under specific evaluation conventions, utility modification may be harmless or disfavored. This supports the narrower formulation used here: self-modification, by itself, is not an impossibility premise.

Proof-carrying code and foundational verification. Proof-carrying code demonstrates that an untrusted producer may supply code together with evidence that a host validates under its own safety policy [8]. Foundational proof-carrying code pushes the trusted base downward toward a smaller set of axioms, a simpler checker, and a smaller runtime substrate [10]. Work on certified self-modifying code further shows that runtime code mutation can be compatible with mechanized formal verification [9]. These results directly motivate the separation between evidence production and control over the validation basis.

Roots of trust, trusted computing bases, and chains of trust. NIST describes roots of trust as highly reliable hardware, firmware, or software components that perform critical security functions and provide a foundation from which trust can be built [12]. NIST platform-resiliency guidance further requires chains of trust to be anchored in roots of trust and protects roots and chains against unauthorized modification [13]. The present paper adopts the same structural lesson without assuming hardware implementation: a persistence claim needs a protected starting basis or a verified invariant, and mutable downstream elements must inherit trust through verifiable transitions rather than through self-assertion.

Reference monitors and security kernels. The reference-monitor tradition requires the policy-enforcement mechanism to be always invoked, tamper-resistant, and sufficiently small or analyzable to be verified [11]. Security kernels implement this concept within a trusted computing base. For persistence assurance, the analogous requirement is that no amendment path capable of changing the objective or its meaning can silently bypass the validation path on which the persistence claim relies.

Remote attestation and evidence appraisal. The IETF RATS architecture separates an Attester that produces Evidence, a Verifier that appraises that Evidence using appraisal policies, reference values, and endorsements, and a Relying Party that acts on Attestation Results [14]. It also emphasizes evidence freshness, binding evidence to the intended target environment, and protection of appraisal policies and reference values. This provides an established engineering vocabulary for the assurance chain used here. Objective persistence is a more specialized claim than platform state attestation, but the separation of measurement, evidence, appraisal, reference values, and reliance is directly applicable.

7 Scope and Limitations

Several limitations are important.

First, the theorem is conditional. It does not state that every self-contained or self-modifying system admits masking amendments. A verified transition system may exclude them.

Second, the theorem concerns amendment-independent assurance, not the empirical probability of objective drift. A system may remain stable for long periods without independent assurance, and it may fail despite apparently independent assurance if the assurance basis is unsound or incomplete.

Third, the result does not establish that any particular human, institutional, hardware, or cryptographic anchor is necessary. Physical externality is neither required nor sufficient. The relevant property is independence from the amendment authority under evaluation, or an invariant strong enough to rule out the relevant masking amendments.

Fourth, the result does not specify a universal reference semantics I_R. In practice, functional equivalence between objectives may itself be difficult to define, especially for learned or distributed

representations. This is a substantive modeling burden rather than a solved component of the framework.

Fifth, the result is not a safety theorem for advanced AI systems. Objective persistence can preserve a harmful objective, and changing an objective can sometimes be desirable. The paper addresses the validity of persistence claims, not whether the persisted objective is aligned, beneficial, or safe.

Sixth, every independence claim is relative to a declared amendment authority A. If the threat model omits a higher-level operator, supply-chain actor, signing-key holder, verifier owner, or other party able to modify the supposed root of assurance, the analysis may overstate independence. A root of assurance is therefore a trust assumption to be minimized, protected, and independently scrutinized, not an automatically trustworthy object.

Seventh, attestation or proof of software state is not automatically proof of semantic objective continuity. Measurements must actually cover the persistence-relevant representation, reference semantics, and update path. A perfectly authenticated measurement of the wrong property does not establish objective persistence.

Eighth, route (i) depends on model adequacy as well as proof soundness. A verified invariant excludes masking amendments only relative to the transition system, semantics, and threat model actually formalized. If the deployed implementation admits transitions omitted from the model, or if the compiler, proof checker, hardware semantics, or other trusted substrate is unsound, the formal result need not transfer to the real system. This is another reason to minimize and expose the trusted base rather than treating "formally verified" as a self-sufficient label.

Finally, the result does not establish empirical prevalence, calibration, or operational thresholds. Its role is conceptual: to separate persistence, pursuit, and assurance, and to identify the structural conditions under which a positive persistence verdict ceases to be independent of the authority capable of changing what that verdict is meant to certify.

8 Conclusion

Objective persistence in self-contained systems should not be framed as a universal impossibility result. Internal components can be immutable, self-modifying systems can be formally verified under suitable models, and evidence may be generated by an untrusted producer while remaining independently checkable.

The narrower structural problem arises when amendment authority extends simultaneously over the objective, the semantics used to identify continuity, and the validation basis that accepts persistence evidence. If that authority admits a masking amendment—a reachable transformation that violates persistence relative to the declared reference semantics while preserving a positive internal verdict—then the verdict is not amendment-independent evidence of persistence.

Defensible assurance therefore has two general routes: constrain the admissible amendment set with a sound invariant that excludes masking transformations, or construct an assurance chain that terminates in a protected root of assurance and preserves state binding, freshness, non-bypassability, and semantic correspondence along the path to acceptance. The protected basis need not be physically external. What matters is that the challenged amendment authority cannot silently rewrite the basis or the chain so as to preserve PASS after violating the declared property.

This reframing preserves the foundational question while aligning it with mature assurance engineering. Objective persistence may be possible, and self-modification may be formally verified, but a persistence claim cannot acquire independent evidential force merely because the same authority that can change the objective can also produce a chain of evidence whose measurement

rules, appraisal policy, reference values, and acceptance criteria it can silently redefine. The strongest formulation is therefore not an "external anchor" theorem but an assurance-independence requirement grounded in protected trust bases, verified invariants, and auditable chains of evidence.

Project Resources

Version-independent Concept DOI: `https://doi.org/10.5281/zenodo.19887869`

Associated LUMINA-30 project resources: `https://github.com/lumina-30/lumina-30-overview`

This paper is supporting research for the existence-condition and assurance layer related to LUMINA-30. It does not define an operational control protocol, certification mechanism, compliance status, or deployment authorization.

References

[1] T. Everitt, M. Hutter, R. Kumar, and V. Krakovna. Reward Tampering Problems and Solutions in Reinforcement Learning: A Causal Influence Diagram Perspective. *Synthese*, 198:6435–6467, 2021. DOI: 10.1007/s11229-021-03141-4.

[2] D. Hadfield-Menell, A. D. Dragan, P. Abbeel, and S. Russell. The Off-Switch Game. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 220–227, 2017. DOI: 10.24963/ijcai.2017/32.

[3] E. Hubinger, C. van Merwijk, V. Mikulik, J. Skalse, and S. Garrabrant. Risks from Learned Optimization in Advanced Machine Learning Systems. *arXiv:1906.01820*, 2019. DOI: 10.48550/arXiv.1906.01820.

[4] V. Krakovna, J. Uesato, V. Mikulik, M. Rahtz, T. Everitt, R. Kumar, Z. Kenton, J. Leike, and S. Legg. Specification Gaming: The Flip Side of AI Ingenuity. Google DeepMind, 2020. `https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/`.

[5] D. Manheim and S. Garrabrant. Categorizing Variants of Goodhart's Law. *arXiv:1803.04585*, 2018. DOI: 10.48550/arXiv.1803.04585.

[6] N. Soares, B. Fallenstein, E. Yudkowsky, and S. Armstrong. Corrigibility. In *Artificial Intelligence and Ethics: Papers from the 2015 AAAI Workshop*, 2015. `https://cdn.aaai.org/ocs/ws/ws0067/10124-45900-1-PB.pdf`.

[7] T. Everitt, D. Filan, M. Daswani, and M. Hutter. Self-Modification of Policy and Utility Function in Rational Agents. In B. Steunebrink, P. Wang, and B. Goertzel, editors, *Artificial General Intelligence: 9th International Conference, AGI 2016, Proceedings*, Lecture Notes in Computer Science, volume 9782, pages 1–11. Springer, 2016. DOI: 10.1007/978-3-319-41649-6_1.

[8] G. C. Necula. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997. DOI: 10.1145/263699.263712.

[9] H. Cai, Z. Shao, and A. Vaynberg. Certified Self-Modifying Code. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 66–77, 2007. DOI: 10.1145/1250734.1250743.

[10] A. W. Appel. Foundational Proof-Carrying Code. In *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 247–256, 2001. DOI: 10.1109/LICS.2001.932501.

[11] National Institute of Standards and Technology. Reference Monitor, *Computer Security Resource Center Glossary*; source: NIST SP 800-53 Rev. 5. `https://csrc.nist.gov/glossary/term/reference_monitor`.

[12] M. Bartock, M. Souppaya, R. Savino, T. Knoll, U. Shetty, M. Cherfaoui, R. Yeluri, A. Malhotra, D. Banks, M. Jordan, D. Pendarakis, J. R. Rao, P. Romness, and K. Scarfone. *Hardware-Enabled Security: Enabling a Layered Approach to Platform Security for Cloud and Edge Computing Use Cases*. NISTIR 8320, 2022. DOI: 10.6028/NIST.IR.8320.

[13] A. R. Regenscheid. *Platform Firmware Resiliency Guidelines*. NIST Special Publication 800-193, 2018. DOI: 10.6028/NIST.SP.800-193.

[14] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, IETF, 2023. DOI: 10.17487/RFC9334.